

Agent eBPF Filter 国创赛 2026 申报书

负责人待填

中国国际大学生创新大赛（2026）项目申报书

一、项目基本信息

字段	内容
项目名称	Agent eBPF Filter：面向 AI Agent 的内核级运行时安全与可观测平台
参赛赛道	高教主赛道（默认）；如学校要求可归入“人工智能+ / 新工科”
参赛组别	创意组（默认，未注册公司）；如已注册企业再调整为创业组
项目类别	新工科 / 人工智能+ / 网络空间安全 / 开发者工具安全
推荐学院	【待填】
项目负责人	【待填：姓名、学号、专业、年级、手机号、QQ、邮箱】
指导教师	【待填：姓名、学院、职称、联系方式】
团队人数	【待填：3-15 人，含负责人】
项目阶段	已有可运行原型：Go 后端 + eBPF 程序 + Vue 前端 + CLI wrapper + Python/Node 适配器

字段	内容
关键词	AI Agent 安全、eBPF、BPF LSM、cgroup、进程树追踪、运行时审计、语义-事实一致性

二、项目简介

Agent eBPF Filter 面向越来越普及的 AI 编码助手、自动化运维 Agent、多 Agent 协同开发环境，提供一个 Linux 本地部署的运行时安全与可观测平台。项目通过 eBPF tracepoints、cgroup/connect、UDP sendmsg、BPF LSM 等内核机制，实时捕获 Agent 及其子进程的执行、文件、网络和策略命中行为；通过 wrapper 与原生 CLI hooks 获取工具调用语义；再由 Go 后端、Vue 前端与执行图谱把“Agent 声称要做什么”和“操作系统实际发生了什么”关联起来，从而实现可观测、可解释、可回放、可策略化控制的 AI Agent 安全平面。

本项目不是简单记录日志，也不是替代大模型本体。它的核心价值在于把 AI Agent 的运行过程从黑箱变成可验证证据链：当一个 Agent 声称只是读取项目文件，却实际启动 shell、访问敏感路径或连接未知外部地址时，系统能够记录完整进程链、文件路径、网络端点和告警原因，并在必要时通过 wrapper、cgroup 或 BPF LSM 进行阻断。

三、项目背景与痛点

3.1 行业背景

AI Agent 正从聊天工具演进为能够调用终端、浏览器、文件系统、MCP 工具和远程开发环境的执行型系统。编码助手可以自动修改仓库、运行测试、安装依赖、访问网络并提交 PR；企业也开始把 Agent 接入研发、运维、数据分析和安全运营流程。Agent 能力越强，运行时风险越接近真实操作系统风险：提示注入、恶意依赖脚本、越权读取密钥、异常外联、下载即执行、多 Agent 文件竞争等问题不再停留在文本层面，而会落到进程、文件和网络行为上。

3.2 外部调研依据

本项目选题不是泛泛讨论“AI 安全”，而是聚焦 Agent 获得本机工具权限后的运行时安全。OWASP LLM06:2025 将 Agent 风险归纳为过度功能、过度权限和过度自治，典型例子是用户只需要读取仓库，第三方扩展却同时具有修改、删除、联网等能力。Five Eyes / CISA / NSA / NCSC 等机构在 2026 年发布的 Agentic AI 采用指南也强调：Agent 会在关键业务中自主规划和执行多步任务，组织需要持续可见性、最小权限、隔离、分阶段上线和运行期保证机制。GitGuardian 2026 secrets sprawl 报告进一步显示，AI 辅助提交和 MCP 配置正在放大密钥泄露风险，开发者本机凭据、MCP 配置和云服务 token 已经成为 Agent 时代的新攻击面。

因此，本项目将用户痛点收敛为三个“不可见”：

1. **Agent 意图不可见**：安全日志知道发生了进程/网络事件，却不知道来自哪个 Agent run、哪个 task、哪个 tool call。
2. **子进程链路不可见**：真正危险行为往往不在 Agent 主进程，而在 shell、python、node、git、npm、curl 等子进程中发生。
3. **阻断效果不可见**：普通审计只能说明“发生过”，不能证明 BPF LSM/cgroup 在内核决策点已经拒绝高风险动作。

3.3 现实痛点

1. **只看模型回答不够**：提示词过滤或回答审查无法证明 Agent 实际执行了哪些命令、子进程和网络连接。
2. **传统日志粒度不足**：普通应用日志往往看不到 shell 派生出的 python、node、git、curl、npm 等子进程，也难以还原进程树与工具调用之间的关系。
3. **传统 EDR 不懂 Agent 语义**：EDR 能看见系统行为，但不知道该行为来自哪个 Agent run、哪个 task、哪个 tool call，也难判断是否偏离用户意图。
4. **阻断路径割裂**：wrapper、hook、网络策略、文件策略常常互相独立，缺少从观测、告警到策略落地的闭环。

5. **复盘困难**：团队缺少可回放的证据链，难以解释一次 Agent 事故中谁触发、哪个工具执行、哪个子进程访问了什么对象。

四、解决方案与产品形态

4.1 总体方案

项目构建 “内核事实层 + 语义声明层 + 关联分析层 + 控制展示层” 的四层架构：

层级	组件	作用
内核事实层	eBPF tracepoints、 cgroup/connect、UDP sendmsg、BPF LSM、 TLS uprobes（显式启用）	获取 exec/open/connect/send/ recv/fork/unlink 等事实事 件，并在特定策略下内核阻 断。
语义声明层	agent-wrapper、 Claude/Gemini/Codex/ Copilot 等 CLI hooks、 Python/Node 适配器	获取 Agent run、task、 tool_call、prompt/respon se 摘要等上层语义。
关联分析层	Go 后端、 EventEnvelope、 semantic_alerts、ML/LL M scoring、执行图	关联进程树、文件、网络、 工具意图，发现语义-事实 不一致与资源浪费。
控制展示层	Vue Dashboard、 Execution Graph、 Network Flow、Config/Policy UI、 OTLP/Prometheus/MCP	展示事件、拓扑、策略、回 放与导出，形成安全运营闭 环。

4.2 核心功能

- **多层级数据捕获**：采集 `execve`、`open/openat`、`connect`、`sendto/recvfrom`、`bind`、`unlink`、`mkdir`、`ioctl` 等系统调用，以及 TCP/DNS/网络流信息。
- **进程树归因**：注册 Agent PID 后，跟踪 `shell`、`python`、`node`、`git`、`npm`、`curl` 等子进程，继承 `agent_run_id`、`task_id`、`tool_call_id`、`trace_id`。
- **BPF LSM 文件/执行阻断**：在 `bprm_check_security`、`file_open`、`file_permission`、`mmap_file`、`inode_*` 等 hook 上按可执行路径、可执行名、文件/目录名进行拒绝。
- **cgroup 网络阻断**：对 cgroup id、PID 所属 cgroup、IPv4/IPv6 地址、TCP/UDP 端口进行内核级拒绝。
- **语义-事实一致性检测**：识别 `SECRET_ACCESS`、`UNEXPECTED_NETWORK_EGRESS`、`UNEXPECTED_CHILD_PROCESS`、`WORKSPACE_ESCAPE`、`RESOURCE_WASTING_LOOP`、`MULTI_AGENT_FILE_CONTENTION` 等告警。
- **执行图谱与回放**：以 Agent Run、Tool Call、Process、File、Network、Policy Decision 为节点，展示 `spawn/open/connect/block` 等边，支持录制、导出和训练样本沉淀。
- **低代码策略编辑**：在前端提供可视化 eBPF/策略构建器，让用户用节点/积木方式组合触发、条件、映射和动作。

五、创新点

5.1 BPF LSM 策略执行形成确定性安全边界

项目把关键阻断能力下沉到 BPF LSM 与 cgroup hook，避免只依赖用户态日志或事后告警。对于禁止执行的二进制、禁止访问的敏感文件名、禁止连接的 IP/端口，系统可以在内核路径返回 `EACCES` 或拒绝 `connect/sendmsg`，让高风险行为在完成前被中止。

5.2 内核态事实与 Agent 语义跨层关联

项目既采集 wrapper/hook 中的 Agent 工具意图，又采集 eBPF 中的真实进程/文件/网络事件。二者通过 agent_run_id、task_id、tool_call_id、trace_id、PID/TGID/PPID、cgroup_id 等字段汇聚到统一 EventEnvelope，从而突破“语义鸿沟”。

5.3 面向多 Agent 的进程树与执行图谱

相比平铺日志，本项目把多 Agent 并发环境中的进程、文件、网络和策略决策建模为图结构。用户可以从一次任务追溯到所有子进程、文件访问、网络连接、阻断动作和告警原因，适合竞赛演示与事故复盘。

5.4 可观测、可回放、可训练的闭环

系统不仅能展示实时事件，还能把行为录制为 JSONL、导出训练样本、进入 ML/LLM 风险评分流程，为后续误报/漏报优化提供数据闭环。

5.5 面向教育与工程实践的可视化策略构建

低代码 eBPF/策略编辑器将复杂内核安全机制转化为可视化节点流，降低教学、演示和团队协作门槛，但在申报叙事中作为配套能力，而非替代核心内核安全创新。

六、技术路线

1. **eBPF 程序开发**：围绕 tracepoints/kprobes/LSM/cgroup hook 编写 C 程序，通过 bpf2go 生成 Go 绑定。
2. **特权后端管理**：Go 后端负责加载、挂载、pin maps/links，消费 ring buffer，并向前端提供 HTTP/WebSocket API。
3. **Agent 接入**：Python/Node 适配器与 agent-wrapper 注册 PID 与上下文；CLI hooks 注入元数据化的 prompt/response 摘要。
4. **事件统一建模**：将内核事件、wrapper 事件、hook 事件转换为版本化 EventEnvelope，并向最近事件、执行图、OTLP/MCP、JSONL 回放输出。

- 5. **策略与告警**：确定性策略负责阻断；语义告警负责解释；ML/LLM 负责离线分析、辅助标注和策略建议，不直接做内核级决策。
- 6. **前端可视化**：Vue 3 前端提供 Dashboard、Network、Execution Graph、Explorer、Hooks、Plugins、ML、Configuration 等页面。
- 7. **验证评估**：通过良性/恶意 replay、os-enforcement-smoke、runtime-benchmark 统计延迟、误报、漏报、阻断成功率和资源开销。

七、项目基础与当前进展

模块	当前进展
后端	已实现 Go HTTP/WS API、eBPF bootstrap、事件归一化、策略配置、Shell session、ML/LLM scoring、OTLP/Prometheus 导出。
eBPF	已覆盖 syscall tracepoints、cgroup sandbox、BPF LSM enforcer、TLS capture opt-in 等路径。
前端	已有 Vue Dashboard、Network Flow、Execution Graph、Explorer、Executor、Hooks、Plugins、ML、Configuration 页面。
Wrapper/Adapters	已有 agent-wrapper、Python/Node PID 注册辅助，支持 CLI hook 与 wrapper 控制。
部署	支持 devcontainer、Makefile、systemd/rc.local 安装、Kubernetes DaemonSet 文档。
验证	已有 Go 单元测试、runtime benchmark、os-enforcement

模块	当前进展	
	preflight/check/smoke 脚本。	
7.1 代码实现与痛点映射		
用户痛点	仓库实现证据	竞赛表达
Agent 子进程链路不可见	backend/event_context.go 的 processContextStore 保存 agent_run_id/task_id/tool_call_id/trace_id，并从父 PID 或 cgroup 继承到子进程。	可以从 Agent Run 追溯到 shell、python、node、git、npm、curl 等实际行为。
只读任务却读密钥/外联	backend/semantic_alerts.go 内置 SECRET_ACCESS、WORKSPACE_ESCAPE、UNEXPECTED_NETWORK_EGRESS、TOKEN_EXFIL_RISK、SEMANTIC_MISMATCH。	能识别“声称只读，实际读取 .env / SSH key 后外联”的偏离行为。
事后日志不能证明阻断	backend/ebpf/lsm_enforcer.c 在 bprm_check_security、file_open、file_permission、inode_* 等 LSM hook 命中后返回 -EACCES。	高风险执行、文件打开、既有 fd 读写、rename/delete 等可在内核决策点被拒绝。
异常外联难以及时控制	backend/ebpf/cgroup_sandbox.c 挂载 cgroup/connect4/connect6/sendmsg4/sendmsg6，支持 cgroup、IPv4/IPv6、	Agent 或其子进程访问未知公网、反连端口时，可在 connect/sendmsg 阶段拒绝。

用户痛点	仓库实现证据	竞赛表达
证据分散难复盘	IPv4-mapped IPv6、TCP/UDP 端口阻断。 backend/event_envelope.go 将 wrapper、native_hook、mcp、process、network、exec、file、policy 归一为 EventEnvelope；backend/execution_graph.go 生成执行图。	答辩可展示 Agent Run → Tool Call → Process → File/Network → Policy 的因果链。
非内核同学难配置策略	frontend/src/components/plugins/recipes.ts 与 transpiler.ts 支持 nc 阻断、SSH 私钥读取保护、敏感 rename 审计等可视化 recipe。	兼顾硬核 eBPF 与教学转化，但可视化只是降低使用门槛，不是核心创新替代品。

八、市场与应用场景

8.1 目标用户

- 高校网络空间安全、软件工程、人工智能实验室：用于 AI Agent 安全教学、科研与竞赛。
- 企业研发效能平台：治理 Codex、Claude Code、Gemini CLI、Copilot 等自动化编码工具。
- 安全运营与合规团队：审计 Agent 读取敏感文件、访问外网、执行危险命令的证据链。
- CTF/攻防演练平台：构建可观测、可阻断、可回放的 Agent 沙箱实验环境。

8.2 价值主张

- 对开发者：知道 Agent 具体做了什么，减少黑箱感。
- 对安全团队：将 Agent 行为转化为可审计证据和可执行策略。
- 对管理者：在不完全禁用 Agent 的前提下实现可控自治，提高工具落地速度。
- 对教育场景：提供 “AI + 系统安全 + eBPF + 工程实践” 的综合项目载体。

九、商业模式初稿

模式	说明
开源核心	保留基础观测、单机部署、教学实验能力，扩大开发者影响力。
专业版订阅	面向实验室/团队，提供多节点聚合、策略模板、报表、SSO、审计留存。
企业私有化	面向企业研发与安全部门，提供 on-prem 部署、定制策略、SIEM/EDR/DevOps 集成。
教育培训	结合 eBPF、AI Agent 安全、竞赛/课程实验，提供教学包和训练营。
咨询服务	为企业建立 Agent 安全基线、红队 replay、策略调优和验收报告。

十、团队分工（提交前补真实姓名）

角色	姓名	主要职责
项目负责人	【待填】	总体架构、答辩、进度管理、竞赛材料统筹。
eBPF/内核开发	【待填】	tracepoints、BPF LSM、cgroup hook、性能优化。
后端开发	【待填】	Go API、事件模型、策略

角色	姓名	主要职责
		引擎、数据存储、测试。
前端开发	【待填】	Vue Dashboard、Execution Graph、低代码策略 UI。
算法与评测	【待填】	语义告警、ML/LLM scoring、benchmark、数据集。
商业与调研	【待填】	市场访谈、竞品分析、商业计划、路演设计。
指导教师	【待填】	技术路线指导、资源协调、申报审核。

十一、实施计划

阶段	时间	目标	里程碑
P0 原型固化	2026.05-2026.06	整理现有功能，形成稳定演示链路	完成申报材料、PPT、5 分钟演示视频。
P1 场景验证	2026.06-2026.08	建立良性/恶意 Agent replay 套件	输出检测率、阻断率、延迟和开销报告。
P2 试点应用	2026.08-2026.10	在实验室/课程/开源社区试点	收集试点反馈、打磨专业版功能。
P3 成果固化	2026.10-2027.03	申请软著/论文/专利，完善文档	形成支撑材料、案例库和商业合作意向。
P4 转化拓展	2027.03-2027.12	推进多节点企业版	完成首批付费试点

阶段	时间	目标	里程碑
		与教育版	或合作协议。

十二、风险与合规

风险	应对
eBPF 需要特权运行	默认本地/实验室部署，服务端启用 token auth，危险功能运行时门控，生产部署保留 systemd/root 边界说明。
TLS 明文捕获涉及敏感数据	明确 opt-in，默认关闭；事件流只保留摘要、长度、角色、厂商等元数据；演示使用脱敏数据。
误报/漏报	建立 replay benchmark，分离确定性阻断与 ML/LLM 建议，保留人工确认。
平台兼容	首期聚焦 Linux；对缺失 tracepoint 做容错；通过 devcontainer 与 Makefile 降低环境差异。
商业数据不足	提交前补充真实用户访谈、问卷、试点证明和可复核财务假设。

十三、预期成果

1. 完成一个可运行的 AI Agent 运行时安全与可观测平台。
2. 形成不少于 3 类演示场景：敏感文件访问、异常网络外联、子进程漂移/下载即执行。
3. 形成测试报告、演示视频、商业计划书、路演 PPT、软著/专利申请材料。
4. 在课程、实验室或企业 PoC 中进行试点，获取反馈证明。
5. 以项目为载体培养团队系统编程、网络安全、AI 工程、产品商业化综合能力。

十四、调研来源与真实性说明

- OWASP LLM06:2025 Excessive Agency：说明 Agent 过度功能、过度权限、过度自治会把提示注入和工具误用放大为真实动作。
- OWASP Agentic AI Threats and Mitigations / Agentic Skills Top 10：强调 agent skills、工具链、记忆、权限清单和多步执行带来的新攻击面。
- Five Eyes / CISA / NSA / NCSC Careful adoption of agentic AI services：强调可见性、最小权限、隔离、运行期保证和持续评估。
- GitGuardian State of Secrets Sprawl 2026：显示 AI 辅助编码、MCP 配置和 AI service key 泄露正在扩大开发者本机与协作链路的密钥风险。
- NIST AI 600-1 / AI RMF：提供 AI 风险治理框架，本项目补充运行时证据与控制落地层。

以上调研均已在 深度调研与代码实现映射.md 中记录，并逐项映射到本仓库代码实现，避免材料停留在概念层。

十五、提交前待补附件

- 【待填】团队成员身份证明/学籍证明。
- 【待填】导师推荐或学院盖章页。
- 【待填】代码仓库链接、commit 截图、开源许可证说明。
- 【待填】系统截图、架构图、测试报告、benchmark 数据。
- 【待填】软著/专利/论文/获奖证明。
- 【待填】用户访谈、问卷、试点意向书或合作证明。